

Evaluación Parcial N°1:

Diseño de Arquitecturas - Grupo Cordillera

Integrantes: Camilo Tapia

Docente: Viviana Poblete

Asignatura: DSY1106 -

Desarrollo Fullstack III

Fecha: 20 de Marzo de 2026

Índice

Contexto y Necesidades del Cliente	1
Arquitectura de Microservicios y Frameworks	2
Ecosistema de Microservicios y Bases de Datos	2
Arquitectura de Comunicación	4
Comunicación Síncrona, Asíncrona y Resiliencia	4
Contenedores y Orquestación	5
Integración con Kubernetes	5
Evaluación del Diseño Propuesto	7
Conclusión	8
Reflexión Personal	8
Bibliografía	9

Contexto y Necesidades del Cliente

El Grupo Cordillera es una organización líder en el sector retail, dedicada a la comercialización de productos con presencia en múltiples sucursales a nivel nacional. Debido a su crecimiento acelerado, operan con una infraestructura tecnológica fragmentada. En la actualidad, la organización opera con sistemas aislados para la gestión de ventas, inventario, finanzas y atención al cliente.

Problemas actuales a resolver: Esta arquitectura descentralizada obliga a los equipos a depender de procesos netamente manuales (extracción de datos desde diferentes plataformas, consolidación en hojas de cálculo, transcripción manual y validación), lo que provoca retrasos críticos y un alto margen de error al momento de consolidar reportes. Esto limita severamente la visibilidad del negocio y afecta la toma de decisiones por parte de la alta gerencia.

Para solventar esta problemática y resolver de raíz las deficiencias del caso, se propone el diseño e implementación de una plataforma analítica basada en una arquitectura de microservicios. Más allá de la integración técnica, el sistema se enfoca en responder a las preguntas críticas del negocio retail: **¿Qué productos son los más vendidos? ¿Qué artículos están estancados en el inventario y deben rematarse con descuento? ¿Qué mercancía está por agotarse y exige emitir órdenes de compra inmediatas?**

Esta solución permitirá automatizar el flujo de la información, administrando inteligentemente el inventario y las compras, garantizando escalabilidad independiente, alta disponibilidad y una operatividad resiliente, apoyados en metodologías de desarrollo Ágil (Scrum) para asegurar entregas continuas de valor.

Arquitectura de Microservicios y Frameworks

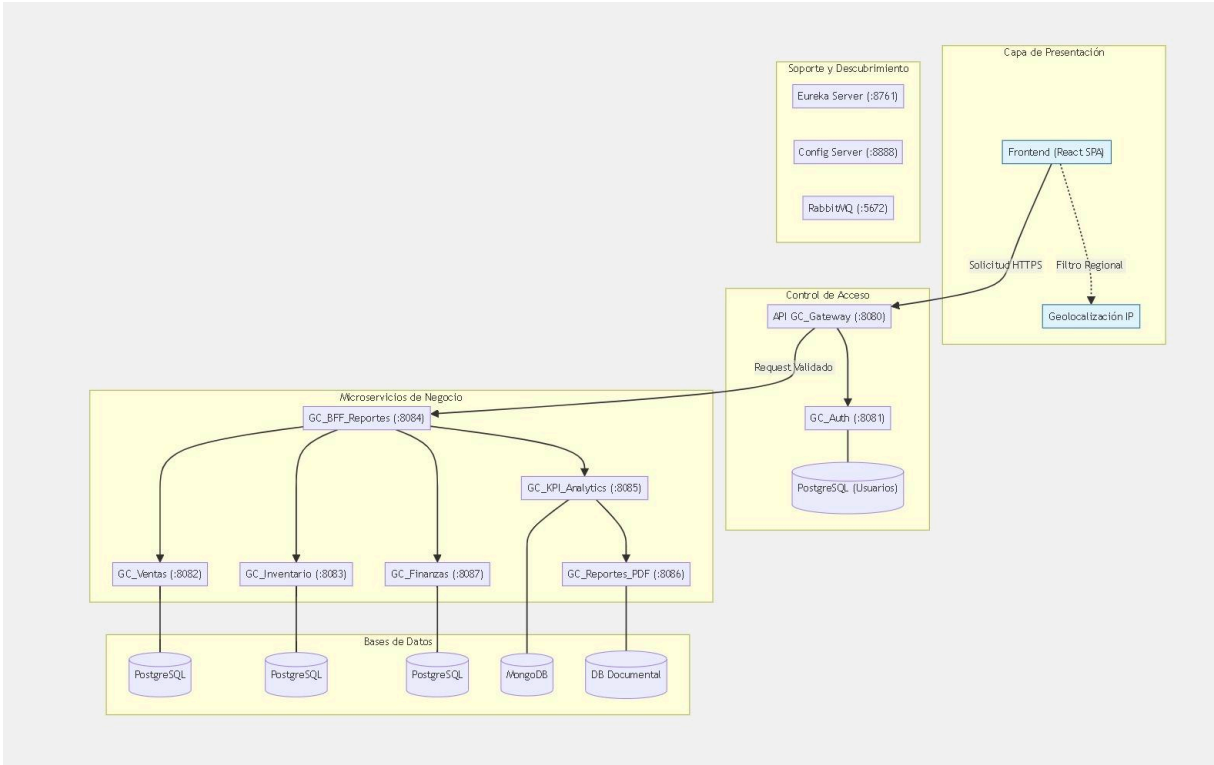
La solución se construirá bajo una arquitectura de microservicios utilizando Spring Boot para el backend, aplicando el patrón MVC (Model-View-Controller) en la estructura interna de cada servicio. El Modelo interactúa con las bases de datos mediante Spring Data JPA (aplicando el Repository Pattern para abstraer la persistencia de datos), el Controlador expone APIs REST, y la Vista será delegada a una aplicación Single Page Application (SPA) desarrollada en React, la cual consumirá estos endpoints para renderizar los dashboards dinámicamente. Además, se utilizará el Factory Method para la instanciación dinámica de objetos complejos, como la creación de distintos tipos de reportes según el perfil del usuario.

Ecosistema de Microservicios y Bases de Datos

Para garantizar el bajo acoplamiento y la seguridad, se definen los siguientes componentes centralizados bajo la nomenclatura del cliente (GC):

- **API GC_Gateway:** Actúa como el único punto de entrada público, encargado de enrutar las peticiones del frontend y validar la seguridad inicial.
- **API GC_Auth (Java):** Gestión de usuarios, roles gerenciales y emisión de tokens JWT. (BD: PostgreSQL).
- **API GC_Ventas (Java):** Consolidación de datos transaccionales de POS y E-commerce. (BD: PostgreSQL).
- **API GC_Inventario_Compras (Java):** Gestión central del stock retail, proveedores y logística. (BD: PostgreSQL).
- **API GC_Finanzas (Java):** Centralización de ingresos y costos. (BD: PostgreSQL).
- **API GC_KPI_Analytics (Python):** Motor analítico para cálculo de indicadores estratégicos y tendencias de negocio. (BD: MongoDB).
- **API GC_BFF_Reportes (Java):** Backend For Frontend dedicado a agrupar los datos de los microservicios y entregarlos optimizados a la interfaz en React. (Caché: Redis, para agilizar consultas recurrentes).

Diagrama de Microservicios



Arquitectura de Comunicación

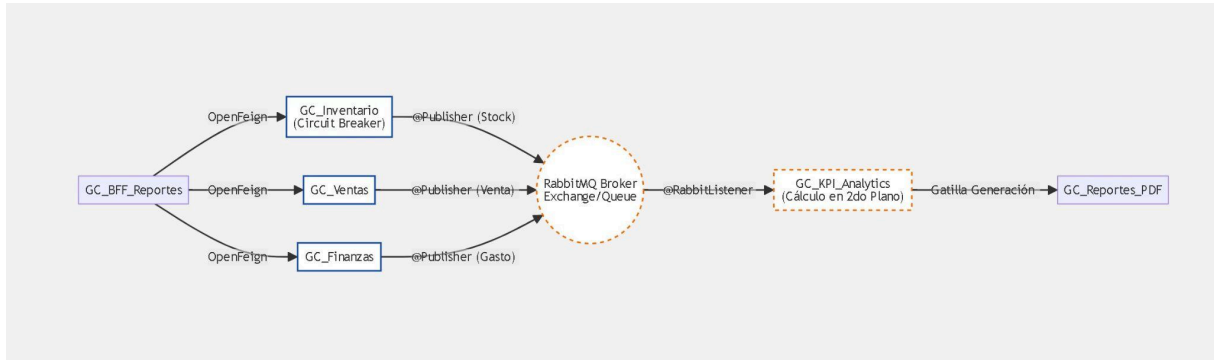
El sistema contará con el **API GC_Gateway** (Spring Cloud Gateway) que actuará como la puerta de entrada. Este componente centralizará el enrutamiento hacia los microservicios internos y validará los tokens de seguridad, ocultando la topología de la red al cliente React.

Comunicación Síncrona, Asíncrona y Resiliencia

Para asegurar el rendimiento y la tolerancia a fallos, se implementan las siguientes estrategias:

- **Comunicación Síncrona (OpenFeign / WebClient):** Se empleará WebClient (reactivo y no bloqueante) y OpenFeign para las consultas de lectura en tiempo real entre el *GC_BFF_Reportes* y los servicios de negocio.
- **Resiliencia (Circuit Breaker):** Se integrará el patrón Circuit Breaker (vía Resilience4j) en las llamadas síncronas. Si un servicio falla o responde lento, el circuito se "abre", evitando que el sistema completo colapse y retornando una respuesta de respaldo.
- **Comunicación Asíncrona (RabbitMQ):** Para operaciones de escritura y actualización de estados, se utilizará un Message Broker (RabbitMQ). Esto garantiza la consistencia eventual; por ejemplo, al registrarse una venta o un movimiento en *GC_Inventario_Compras*, se emite un evento asíncrono que *GC_KPI_Analytics* consume en segundo plano para recalcular métricas sin bloquear el flujo principal.

Diagrama de Comunicación y Resiliencia



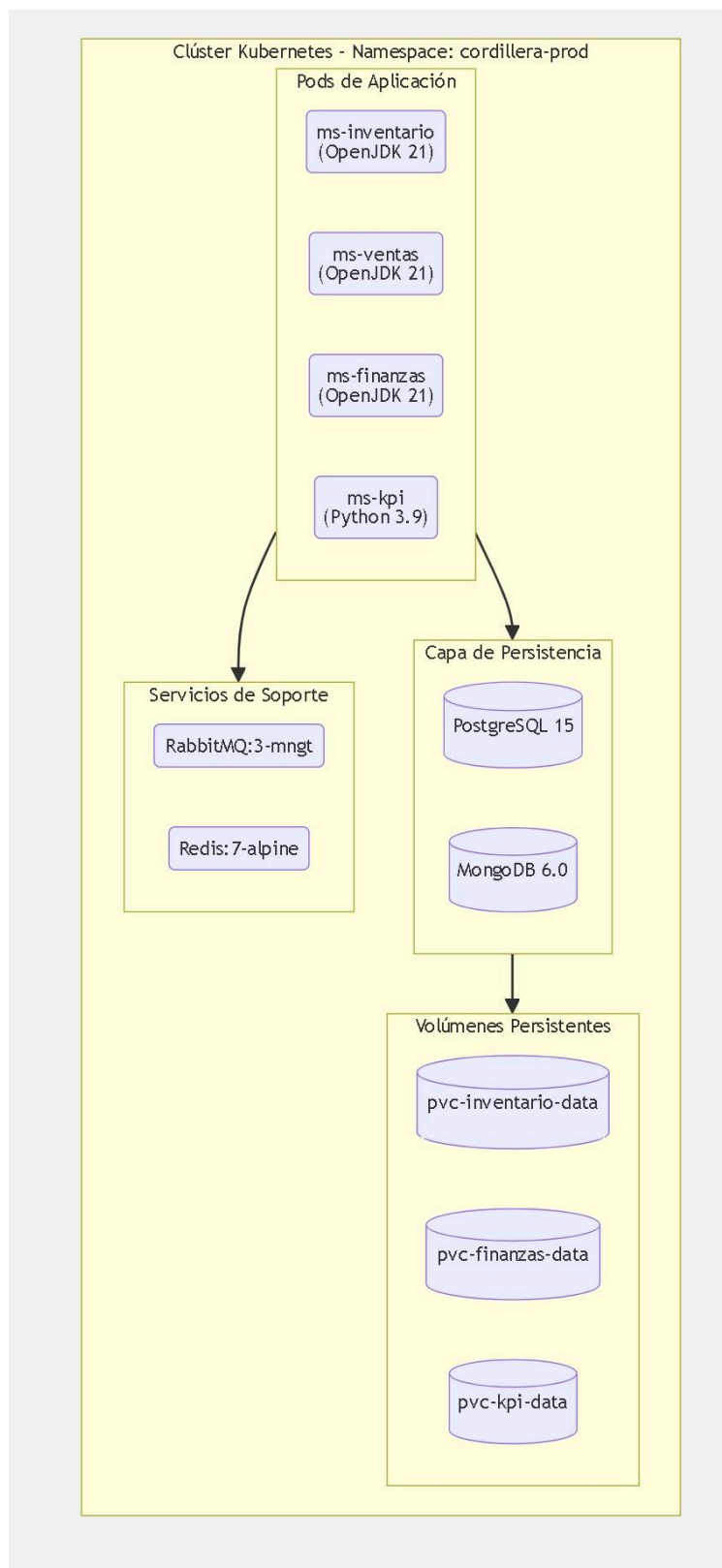
Contenedores y Orquestación

La contenerización es un pilar fundamental en este ecosistema híbrido. Se utilizará **Docker** para empaquetar de forma aislada cada microservicio (tanto los entornos Java como Python), las bases de datos y las herramientas de infraestructura junto con sus dependencias. Esto elimina las inconsistencias entre entornos y estandariza los despliegues.

Integración con Kubernetes

Todo el ecosistema de contenedores será orquestado mediante **Kubernetes (K8s)**. K8s administra el ciclo de vida de los contenedores, aportando eficiencia mediante autorreparación y auto-escalamiento horizontal. Esto es vital para la arquitectura políglota: permite escalar de forma independiente levantando más réplicas del contenedor de **GC_KPI_Analytics** (Python) y **GC_Reportes_PDF** exclusivamente durante fin de mes o campañas masivas de retail, cuando la gerencia exige cálculos intensivos.

Diagrama de Infraestructura Docker/Kubernetes



Evaluación del Diseño Propuesto

Resolución de Dificultades: Esta arquitectura elimina la transcripción propensa a errores. Responder qué productos liquidar o cuáles reabastecer pasa de tardar días (con procesos manuales en Excel) a resolverse en milisegundos.

Seguridad y Privacidad: El sistema protege los datos transaccionales mediante segmentación de redes. El *GC_Auth* y el *GC_Gateway* aseguran que solo personal autorizado acceda mediante tokens JWT, segmentando la visibilidad de los KPIs según los roles gerenciales.

Sostenibilidad y Escalabilidad: El uso de microservicios y Kubernetes asegura la sostenibilidad a largo plazo optimizando los recursos de hardware, escalando solo los módulos necesarios y reduciendo costos operativos en la nube.

Ventajas: Alta resiliencia operativa, eficiencia analítica insuperable gracias a Python, eliminación de procesos manuales y un historial inmutable de reportes mensuales en PDF para auditoría.

Desventajas: La integración de un ecosistema Frontend (React) con un backend políglota (Java + Python) incrementa la complejidad del despliegue inicial y requiere un equipo técnico multidisciplinario para gestionar redes distribuidas y consistencia eventual de datos.

Conclusión

El diseño de esta arquitectura de microservicios para Grupo Cordillera resuelve la problemática inmediata de la fragmentación de datos y los procesos manuales, estableciendo una base tecnológica escalable y preparada para el futuro. La aplicación de patrones de diseño, junto con contenedores Docker y orquestación, asegura que la organización pueda tomar decisiones estratégicas de retail basadas en datos en tiempo real. Esto transforma al departamento de TI en un habilitador clave del negocio, entregando valor continuo mediante prácticas ágiles y garantizando que el sistema pueda crecer orgánicamente sin comprometer su rendimiento ni seguridad.

Reflexión Personal

Como reflexión final, considero esta primera parte como el cimiento crítico para el desarrollo de nuestro caso semestral. Más allá de definir tecnologías, esta etapa me permitió comprender que los problemas reales de una empresa —como los cuellos de botella generados por consolidar reportes en Excel— no se solucionan simplemente escribiendo código, sino diseñando ecosistemas inteligentes. Entender cómo patrones como el BFF, el uso de RabbitMQ para la asincronía y la orquestación con Kubernetes trabajan juntos, me ha demostrado que una buena arquitectura es la clave para entregar una solución que no solo funcione, sino que sea resiliente, escalable y aporte un valor estratégico real al directorio de Grupo Cordillera.

Bibliografía

- *Evaluación Parcial N°1 Diseño de Arquitecturas: Caso Semestral Grupo Cordillera*. Subdirección de Diseño Instruccional.
- Fowler, M. (2024). *Microservices Resource Guide*. MartinFowler.com.
<https://martinfowler.com/microservices/>
- Meta Platforms, Inc. (2025). *React: The library for web and native user interfaces*.
<https://react.dev/>
- Newman, S. (2024). *Building Microservices: Designing Fine-Grained Systems* (3.^a ed.). O'Reilly Media.